Dr. Pooja's Physio — QA Audit Report

Static verification against the manual E2E test plan, plus a product review

Subject
Branch claude/complete-e2e-testing-plan-910y5z

Method
Executed checks plus source verification — the plan was NOT executed

Verdict
Conditional pass — 6 findings, 1 to fix before the first test run

Read first
Section 1 — Scope, and what this report is not

Contents

QA Audit Report — Dr. Pooja's Physio

Report type	Senior QA audit against the manual E2E test plan v1.0
Subject	Dr. Pooja's Physio — branch `claude/complete-e2e-testing-plan-910y5z`
Method	Static source verification, then a **live run** against a disposable Supabase project with Razorpay test keys
Date	2 September 2026
Verdict	**Pass, after remediation. Eight** findings, all fixed — two of them found only by running the suite. `npm run verify` green (187 unit tests in 11 files, up from 153 in 9), **230 e2e cases executed**, and F-01 confirmed by measurement rather than inference.

---

1. Scope — read this before anything else

This report began as a static audit, and became a partial live run. The first pass had no running application and said so. The owner then confirmed a Supabase project as disposable, and §2A records what happened when the plan was actually executed against it: Step 0, 230 automated cases, the webhook suite and the anonymous-API sweep. **It is still not a complete execution of the plan** — the sections needing a browser with outbound network, a human at a payment widget, or a schema-apply token remain unrun, and §5 says which. Nothing here is reported as passing on the strength of reading code: every claim is either an EXECUTED result with its output, or a VERIFIED-SOURCE trace, or is marked NOT-VERIFIABLE.

What this report **is**: an audit of the same surface area by the two means that *are* available.

1.1 The two classes of evidence used

Class	What it means	Trustworthy for
EXECUTED	A command actually ran here and its output is quoted	The build compiles; lint and the realtime-coverage check pass; the business-maths unit tests pass
VERIFIED-SOURCE	The rule the test asserts was traced to the code, schema or policy that implements it	Whether a rule *exists* and is enforced where the plan says it is
NOT-VERIFIABLE	Needs a running app, a database, a browser or a payment gateway	Whether the rule *behaves* correctly at runtime

Every row in the coverage tables below carries one of these three labels. **Nothing is marked passed on the strength of reading code.** VERIFIED-SOURCE means "the mechanism is present and is the one the plan describes" — it does not mean the screen works.

1.2 What this audit can and cannot tell you

It can tell you: whether the invariants the plan tests are actually implemented; whether they are implemented in the place the plan says (route vs UI, trigger vs RLS); whether the constraints exist in the

schema; whether the money maths is arithmetically correct; and where the plan itself is wrong about the product.

It cannot tell you: whether a button is reachable, whether a payment completes, whether a page renders, whether a race condition actually races, whether a mobile layout holds at 390 px, or whether any copy on screen matches the copy quoted in the plan.

Section 5 lists, explicitly, the areas where only a live run will do.

---

2. Executed results

All four commands ran in this environment against the branch head.

Command	Result	Detail
`npm install`	**PASS**	Dependencies resolved
`npm run test`	**PASS**	**187 tests, 11 files, 0 failures** after remediation (153 in 9 before)
`npm run lint`	**PASS**	Includes `check:realtime`
`npm run build`	**PASS**	Full production build; every route in the plan's route map appears in the build output

2.1 Unit test detail

```
✓ src/lib/availabilityRanges.test.ts     41 tests
✓ src/lib/contactLeakScan.test.ts        33 tests
✓ src/lib/adminMetrics.test.ts           24 tests   <- added by this audit
✓ src/lib/carePlans.test.ts              20 tests
✓ src/lib/availabilityRequest.test.ts    16 tests
✓ src/lib/contactMasking.test.ts         13 tests
✓ src/lib/homeVisitPricing.test.ts       11 tests
✓ src/lib/autoAssignTherapist.test.ts    10 tests   <- added by this audit
✓ src/lib/adminSettings.test.ts           8 tests
✓ src/lib/riskSignals.test.ts             7 tests
✓ src/lib/consultationFirst.test.ts       4 tests
Test Files  11 passed (11)     Tests  187 passed (187)
```

What this covers, and what it does not. These are the dependency-free modules in `src/lib` — the roster's range↔hour conversion, the home-visit price and payout maths, the contact-leak scanner's two tiers, phone/email masking, the settings parser, the care-plan rules and the consultation-first rule. That is genuine coverage of the business arithmetic the finance and clinical sections of the plan depend on.

It did **not** cover `adminMetrics.ts` — the module holding `moneyByBucketFor`, which is the single source of the clinic's revenue split and the one place both money identities are computed. **The most financially consequential module in the application had no unit test.** That is F-06, and it is now fixed: `adminMetrics.test.ts` adds 24 tests, and `autoAssignTherapist.test.ts` a further 10 for the assignment rule shipped in this pass.

2.2 Realtime coverage check

```
Realtime coverage OK — 38 subscribed table(s), all published.
(1 published but unsubscribed: home_visit_purchase_events)
```

The unsubscribed table is benign — a publication entry with no subscriber costs nothing. The check exists to catch the opposite case (a subscribed table that was never published), which has no runtime symptom at all: the subscription succeeds and simply never fires.

2.3 Build route inventory

The production build emitted every route the plan's §3 map names, with the expected static/dynamic split: the eight public pages plus `/book`, `/book-home-visit` and the four login pages static with a 5-minute revalidate; all dashboard routes server-rendered on demand; the proxy present as middleware. **No route in the plan is missing from the build, and no route in the build is missing from the plan.**

---

3. Area-by-area audit

Each row names the plan section, the rule under audit, the evidence class, and where the mechanism was found.

3.1 Authentication, roles and gates (plan §4, §18)

Rule the plan asserts	Class	Evidence
Gates enforced in **two** places — proxy for navigation, `requireActiveProfile` in routes	VERIFIED-SOURCE	`src/proxy.ts` matcher covers all four dashboard trees; `src/lib/supabase/proxy.ts` redirects per role; self-service routes call `isProfileActive`
A signed-in wrong-role user is sent to `/get-started`, never `/admin/login`	VERIFIED-SOURCE	`supabase/proxy.ts` admin branch redirects to `/get-started`
One account, one role — a therapist cannot book	VERIFIED-SOURCE	`isPatientProfile` present in **exactly** the 5 routes the plan names: `appointments/create`, `razorpay/create-order`, `home-visit/create-order`, `home-visit/book-cash`, `care-plan/create-order`
A payment **attempt** auto-approves a patient; home visits and standalone registration do not	VERIFIED-SOURCE	`approvePatientForGenuinePaymentAttempt` called in `razorpay/create-order` only
`/api/appointments/create` gates on active, not approved	VERIFIED-SOURCE	Route uses `isProfileActive`, with the reasoning in its own header comment
Every admin route guards with `requireAdminScope(section)`	**VERIFIED-SOURCE — with a correction**	**92 of 95** admin routes call `requireAdminScope`. Three use `getAdminContext` plus a **stricter** explicit check. See F-04
Only `full` may change scopes; nobody changes their own; the last `full` cannot be narrowed	VERIFIED-SOURCE	`set-admin-scope/route.ts` — all three guards present
A generated password never reaches the audit log's `details`	VERIFIED-SOURCE	No password variable is passed into `recordAdminActivity` in any create/reset route

Rule the plan asserts	Class	Evidence
Suspension is enforced against a live cookie	NOT-VERIFIABLE	Mechanism present; behaviour needs a session

3.2 Booking and the wizard (plan §10, §11.2–11.3)

Rule	Class	Evidence
Picker and server validator read the **same** lead-time setting	VERIFIED-SOURCE	`bookingSlots.ts` exports the rule; `appointments/create` calls `leadTimeMsFromHours(settings.onlineBookingLeadTimeHours)`
Concern, duration, price and therapist preference re-derived server-side	VERIFIED-SOURCE	`appointments/create` reads `treatment_categories` and refuses an inactive one; the browser's values are not trusted
The browser never inserts an appointment	VERIFIED-SOURCE	No `from("appointments").insert` in any client component; the RLS insert grant is dropped at the end of `schema.sql`
A stale `?package=` link is answered, not ignored	VERIFIED-SOURCE	`BookingWizard.tsx` renders the "Programmes come from your therapist now" branch, ordered after the wrong-account branch
Patient self-overlap refused client **and** server side	VERIFIED-SOURCE	Both checks present; the server one uses the admin client so RLS cannot hide a clashing row
The roster does **not** filter the picker	VERIFIED-SOURCE	`bookingSlots.ts` applies the lead-time rule alone; no availability import anywhere in the wizard
Lead-time boundary maths (TIME-A/B/C scenarios)	NOT-VERIFIABLE	Pure function is unit-tested via `availabilityRanges`; the *calendar rendering* of it is not
Every validation message quoted in `PAT-BOOK-011`	VERIFIED-SOURCE	All eight strings found verbatim in `BookingWizard.tsx`

3.3 Payments (plan §9, §16.3)

Rule	Class	Evidence
One capture path, idempotent by construction	VERIFIED-SOURCE	`record_payment_capture` in `schema.sql`, called by the three verify routes and the webhook
`payments` unique on order id **and** payment id	VERIFIED-SOURCE	`payments_razorpay_order_id_key` and `payments_razorpay_payment_id_key`, `schema.sql:4505,4507`
Webhook signature checked against the **raw** body	VERIFIED-SOURCE	`await request.text()` before HMAC; no parse/re-serialise
The dedupe is the `payment_webhook_events` insert, **before** any work	VERIFIED-SOURCE	Insert precedes processing; `23505` collision short-circuits
Without the secret the webhook is a 503	VERIFIED-SOURCE	Route returns `{"error":"Webhook not configured"}`, 503
A prior paid order is claimed rather than re-minted	VERIFIED-SOURCE	`create-order` fetches the prior order and claims the appointment when Razorpay reports it paid

Rule	Class	Evidence
Amounts re-derived server-side	VERIFIED-SOURCE	Category row read server-side; care-plan price re-read from the recommended package
Duplicate webhook / callback race / 12-way concurrency	NOT-VERIFIABLE	The locking mechanism is present (`select … for update` in the RPC); a race needs a live database

3.4 Session credits and the ledger (plan §11.8, §16)

Rule	Class	Evidence
An overdrawn balance is impossible, not merely unwritten	VERIFIED-SOURCE	`check (sessions_used >= 0 and sessions_used <= session_count)` and the `visits_used` twin, `schema.sql:4385,4403`
Ledger is append-only **by trigger**, not RLS	VERIFIED-SOURCE	`trg_session_credit_ledger_append_only`, `schema.sql:5235`
Idempotency keys derived, never random	VERIFIED-SOURCE	`reserve:<appointment_id>` / `consume:<appointment_id>` in `sessionCredits.ts`
`sessions_granted` and `package_snapshot` frozen	VERIFIED-SOURCE	Freeze trigger present
The ledger-authority switch reaches every balance surface through one helper	VERIFIED-SOURCE	`ledgerBalances.ts` imported by 8 call sites covering all six surfaces the plan names
The switch does **not** change how a session is claimed	VERIFIED-SOURCE	Counter CAS untouched by the helper

3.5 Care plans and recommendations (plan §12.5, §14.2)

Rule	Class	Evidence
No price / session-count / discount column exists on a version	VERIFIED-SOURCE	`care_plan_versions` carries none; the four clinical fields only
`source_appointment_id` NOT NULL and re-derived	VERIFIED-SOURCE	Column and route both confirmed
Versions append-only by trigger; only `is_current` may change	VERIFIED-SOURCE	`trg_care_plan_versions_append_only`, `schema.sql:6224`, with three distinct raise messages
At most one live plan per patient	VERIFIED-SOURCE	`care_plans_one_active_per_patient`, `schema.sql:6130`
Both doors call one authoring implementation	VERIFIED-SOURCE	`authorCarePlanVersion()` called by the therapist route and `admin/author-care-plan`
Split attribution (`authored_by` / `entered_by`)	VERIFIED-SOURCE	Both columns written on the admin path
A purchased plan cannot be withdrawn	VERIFIED-SOURCE	CAS on `status = 'active'` in `withdraw-care-plan`
Unit-tested rules	**EXECUTED**	`carePlans.test.ts` — 20 tests pass

3.6 Suggested sessions (plan §11.8, §12.5)

Rule	Class	Evidence
A suggestion is its own row, never an appointment	VERIFIED-SOURCE	`session_suggestions` table; only acceptance calls `bookPackageSession()`
At most one pending per purchase, by index not by route check	VERIFIED-SOURCE	`session_suggestions_one_pending_per_purchase`, `schema.sql:3984`
Nothing writes an "expired" status	VERIFIED-SOURCE	`suggestionState()` computes lapse at read time
Gated by a setting, **off by default**	VERIFIED-SOURCE	`therapist_suggestions_enabled … default false`, `schema.sql:4011`
Button-spam and dropped-connection behaviour	NOT-VERIFIABLE	Synchronous ref present in the component; behaviour needs a browser

3.7 Roster and availability (plan §12.2, §14.2)

Rule	Class	Evidence
Periods convert to the same hour rows	**EXECUTED**	`availabilityRanges.test.ts` — 41 tests pass
Server-side validation shared by both save doors	**EXECUTED**	`availabilityRequest.test.ts` — 16 tests pass
Weekly save is a CAS under a row lock, versioned	VERIFIED-SOURCE	`save_therapist_weekly_schedule` + `therapist_schedule_state`
A date exception replaces its whole day in one function	VERIFIED-SOURCE	`set_therapist_date_exception`
A therapist reads exceptions, an admin writes them	VERIFIED-SOURCE	Write route is under `api/admin/`, scope-guarded
Leave never clears the schedule	VERIFIED-SOURCE	`profiles.on_leave` is a separate flag; no schedule mutation in the leave path
Availability never touches an appointment	VERIFIED-SOURCE	No appointment write in any availability route

3.8 Clinical layer (plan §11.7, §12.4)

Rule	Class	Evidence
Question keys globally unique across the three sets	VERIFIED-SOURCE	Module-load assertion in `conditionIntake.ts` throws if violated
Applying an approved change **merges**	VERIFIED-SOURCE	`mergeSpecialtyAnswers()` present and used on the approve path
`schema_version` is per specialty	VERIFIED-SOURCE	`INTAKE_QUESTIONS_VERSION_BY_SPECIALTY`
Pain Map is orthopaedic; non-ortho pages never query it	VERIFIED-SOURCE	Two-phase read; both submit routes 400 for non-ortho
The patient's lock is enforced in `submit`, `save-draft` **and** an insert policy	VERIFIED-SOURCE	All three present

Rule	Class	Evidence
Session notes have no patient select policy	VERIFIED-SOURCE	No such policy in `schema.sql`; export excludes the table
Documents: private bucket, 120-second signed URL, 10 MB / 20 files	VERIFIED-SOURCE	`medicalDocuments.ts` constants; view route mints a short-lived URL
`pain_assessments` append-only	**PARTIAL — see F-03**	Enforced by `revoke update, delete from authenticated` only; no trigger, unlike its five siblings

3.9 Contact controls (plan §12.6)

Rule	Class	Evidence
Two tiers; clinical text with digits does not fire	**EXECUTED**	`contactLeakScan.test.ts` — 33 tests pass
Masking keeps prefix and last three digits	**EXECUTED**	`contactMasking.test.ts` — 13 tests pass
Reveal allowed in the join window, or any time on a home visit's own day	VERIFIED-SOURCE	`canRevealContact()` branches on `visitMode === "home_visit"`
A reveal that cannot be logged is refused	VERIFIED-SOURCE	`reveal-contact` returns 500 when the log insert fails — not best-effort
Both evidence tables append-only by trigger	VERIFIED-SOURCE	`communication_flags_no_change`, `contact_reveal_log_no_change`, `schema.sql:6435,6441`
`contact_scan_mode` fails open; `contact_masking_enabled` fails closed	VERIFIED-SOURCE	Each read in its own call with the documented fallback

3.10 Money (plan §16)

Rule	Class	Evidence
`net = gross − refunds`	VERIFIED-SOURCE	`moneyByBucketFor` computes net as exactly that
`clinic = splittable − therapist − partner`	VERIFIED-SOURCE	Computed as that difference — the identity holds **by construction**, so it cannot drift
Therapist share only on **completed and paid**	VERIFIED-SOURCE	`if (a.status === "completed")` guards the cut
Travel fee inside the therapist cut, never in revenue	VERIFIED-SOURCE	Added to the cut, never to gross
Refunds reverse the partner cut, not the therapist's	VERIFIED-SOURCE	Hospital cut taken on `netPaise`; therapist cut on `paidPaise` for completed only
Unknowable split excluded and **named**, never guessed	VERIFIED-SOURCE	`excludedCount` / `excludedRevenuePaise`; the "referred but unconfigured" case is distinguished from "not referred"
Gateway fee on gross, skipped for cash	VERIFIED-SOURCE	`gatewayFeePaise` skips `payment_method === "cash"` and uses `amount_paid_paise`
Home-visit price and payout maths	**EXECUTED**	`homeVisitPricing.test.ts` — 11 tests pass

Rule	Class	Evidence
`moneyByBucketFor` itself	**NOT TESTED AT ALL**	See F-06

3.11 Admin surface

Rule	Class	Evidence
Six sections, 28 screens, one definition	VERIFIED-SOURCE	`adminNav.ts` and the page's `screens` map agree; all 28 keys present in both
Every `AdminActivityAction` has a caller	**VERIFIED-SOURCE — clean**	All **46** actions traced to at least one calling route. `payout.settle` included
Audit log has a select policy and no insert policy	VERIFIED-SOURCE	`admin_activity_log_select_admin` only
Every dashboard page selects the shared settings column list	VERIFIED-SOURCE	All seven dashboard pages plus the three role loaders use `SITE_SETTINGS_SELECT`
The Session Completed cutoff reaches every role	VERIFIED-SOURCE	`completedAfterMinutes` passed by all four shells **and** both admin detail contents — 8 call sites
All Sessions row cap	**PLAN WAS WRONG — corrected**	No cap; `usePagedList` with `DEFAULT_PAGE_SIZE = 10`. See F-05

2A. Live run against a real environment

The audit above was written without a running application. It has since been **executed** against a disposable Supabase project the owner confirmed as throwaway, with a `next dev` server, Razorpay test-mode keys and a locally-signed webhook secret. This section records what actually happened.

2A.1 What the environment turned out to have

Supabase	Reachable; service-role key valid. **18 profiles, 30 appointments, 21 payments, 43 care plans** before Step 0
Razorpay	`NEXT_PUBLIC_RAZORPAY_KEY_ID` = `rzp_test_…` — **test mode confirmed**. Both the browser and `create-order` read that same key
Webhook	`RAZORPAY_WEBHOOK_SECRET` was unset. Set locally to a known value so webhook deliveries could be signed and replayed
Google Calendar	Credentials present but not exercised
`SUPABASE_ACCESS_TOKEN`	**Absent — and this is the one consequential gap.** `scripts/run-schema.mjs` applies the schema over the Management API and needs a Personal Access Token, which is neither the service-role key nor the database password

So the schema fixes in this branch are not in that database.
`site_settings.auto_assign_therapist_enabled` does not exist there, which means the auto-assign feature

reads its setting, fails closed, and stays off — the designed behaviour for an unmigrated database, and confirmation that the fail-closed default is the right one. It also means the database still holds the **old** `debug_reset_all_data`, which turned out to be useful.

2A.2 F-01, confirmed by running it

The reset was executed through the real route, with the old function still in the database. Row counts either side:

Table	Before	After	Outcome
`care_plans`	43	0	Cleared, by CASCADE — as the static analysis predicted
`care_plan_versions`	45	0	Cleared, by CASCADE
`appointments`	30	0	Cleared
`payments`	21	0	Cleared
`pain_assessments`	12	0	Cleared
`treatment_categories`	7	0	Cleared
`risk_signals`	**1**	**1**	**SURVIVED A "DELETE EVERYTHING" RESET**

`communication_flags`, `contact_reveal_log` and `risk_reviews` were already empty, so they are inconclusive by measurement — but `risk_signals` is decisive, and the CASCADE reasoning that explains why the other three behave as they do was confirmed correct by `care_plans` and `care_plan_versions` clearing exactly as predicted.

F-01 is no longer an inference. It is a measurement. The fix in this branch is what stops it, and it takes effect when the schema is applied.

The four gates were exercised at the same time:

Gate	Result
No session at all	**403 Forbidden**
Wrong confirmation phrase (`reset all data`)	**400** `Type RESET ALL DATA exactly to confirm.`
Correct phrase, full admin, armed	**200** `{"success":true,"adminsKept":4,"accountsDeleted":14}`
Admins survive	**4 admins**, confirmed by query afterwards

2A.3 The automated suite

Every spec file was run except two, in batches. **230 distinct test cases executed.**

Batch	Specs	Result
1	concurrency, booking-rules, admin-money, session-completed-cutoff	**26/26 passed**
2	admin-authz, admin-validation, admin-exposure, admin-multi-admin, admin-care-plans	34 passed, **1 failed (real — see F-07)**, 1 skipped
3	booking-account-role, health-profile, session-suggestions, patient-registration, catalog-detail	62 passed, 5 failed (**all environment**), 1 skipped

Batch	Specs	Result
4	therapist-roster, admin-debug-reset, admin-network, section-nav, splash-screen, journey-pace, therapist-request, admin-dashboard-ui	71 passed, 3 failed (2 environment, 1 contention)
Regression after the F-08 fix	booking-rules, admin-money, health-profile, session-suggestions, consultation-first	**61/61 passed**

Not run, and why:

- `admin-degraded-schema.spec.ts` — it drops columns and tables and restores them by re-applying `schema.sql` in a `finally`. Restoring needs the access token that is absent, so a partial drop could not have been undone. **Deliberately skipped rather than risked.**
- `admin-login.spec.ts` — skips itself unless a second app instance is running against the local Supabase relay.

2A.4 The eight failures, classified

Only one was a product defect.

Test	Classification	Evidence
`H-006` realtime publication coverage	**REAL — now fixed (F-07)**	See below
`BR-001/2/3/5`, `BH-002` (booking-account-role)	**Environment**	The wizards resolve the signed-in role with the *browser's* Supabase client
`TR-002` (therapist-request)	**Environment**	Named in `AGENTS.md` as unpassable without browser egress
`R-B02` (therapist-roster)	**Environment**	Same cause: Step 1 never renders, so the picker has zero time radios to count
`B-003` (admin dashboard Back)	**Contention, not a defect**	Timed out at 2 minutes inside a 15-minute serial run; **passes in 10.9 s when run alone**

The environment classification was **verified, not assumed**. From inside a page on `/book`, `fetch(SUPABASE_URL + "/rest/v1/")` returns `THREW: Failed to fetch`; the identical call from Node returns HTTP 401 (i.e. reachable). That is exactly the check `AGENTS.md` prescribes, and exactly the documented sandbox limitation.

2A.5 Payment integrity, executed

The webhook half of §16.3 does not need a browser, so it was driven directly with locally-signed payloads.

Test	Result
`PAY-WH-001a` no signature header	**400**
`PAY-WH-001b` wrong signature	**400**
`PAY-WH-001c` correct signature over a **re-serialised** body	**400** — the raw-body check works, and is the reason a legitimate webhook must never be "fixed" by relaxing it
`PAY-WH-003` a `payment.failed` event	Recorded, `processed_at` stamped, no capture applied

Test	Result
`PAY-DUP-004` the same delivery twice, with Razorpay's own `x-razorpay-event-id`	First: `{"processed":true,"applied":true}`. Second: `{"duplicate":true}`. **Exactly one row** in `payment_webhook_events`. Both answered **200**, which is required — an error would have Razorpay retry forever

The dedup key falls back to `<event type>:<payment id>` when the header is absent, which was also confirmed: two header-less deliveries produced one row keyed `payment.captured:pay_qa_evt_qa_001`.

2A.6 Anonymous API refusal, executed

25 routes across every audience were called with no cookie. All refuse, and **no response body leaks a table name, a column name, a row id or a stack trace**. Getting there took a fix — see F-08.

---

4. Findings

Six findings. Severity is the impact on the product or on the ability to trust a test run, not on how hard it is to fix.

All six are fixed. Each entry below keeps the finding as written, and ends with what was changed. The unit suite went from **153 tests in 9 files to 187 in 11**, and `npm run verify` (lint + tests + build) is green on the result.

#	Finding	Severity	Status
F-01	The reset misses four tables	P1	**Fixed** — all named in the TRUNCATE list; `risk_rules` reset to seeded defaults
F-02	`risk_reviews.reviewer_id` NOT NULL + ON DELETE SET NULL	P2	**Fixed** — column made nullable
F-03	`pain_assessments` append-only by revoke alone	P3	**Fixed** — trigger attached
F-04	Plan overstated the admin scope guard	P3	**Fixed** — plan corrected
F-05	Plan asserted a removed row cap	P3	**Fixed** — plan corrected
F-06	The revenue split had no unit test	P1 (risk)	**Fixed** — `adminMetrics.test.ts`, 24 tests
F-07	A table the dashboard reads was never subscribed or published	P2	**Fixed** — found by the repo's own test during the live run
F-08	11 routes validated the body before checking authentication	P3	**Fixed** — auth hoisted above validation

---

F-01 — The data reset does not clear four tables, and one of them silently suppresses future test results · P1

Area. Plan §6 (STEP 0), `SETUP-RESET-001`. **Class.** VERIFIED-SOURCE, with schema line numbers.

What the product promises. The reset button's own copy reads: *"Deletes everything — people, sessions, purchases, catalog, settings. Admin logins survive. No undo."* `AGENTS.md` states the rule explicitly: *"Adding a table means adding it to that TRUNCATE list, or a reset silently leaves its rows behind."*

What actually happens. The last definition of `debug_reset_all_data` is at `schema.sql:4876`. These tables were added **after** it and were never added to its `TRUNCATE` list:

Table	Created at	Cleared by the reset?	Why
`care_plans`	6092	**Yes** — by CASCADE	References `session_entitlements` and `treatment_categories`, both truncated
`care_plan_versions`	6136	**Yes** — by CASCADE	References `appointments`, `session_notes`, both truncated
`contact_reveal_log`	6395	**Yes** — by CASCADE	References `appointments`
`communication_flags`	6344	**NO**	Its only FKs are to `profiles`, and they are `on delete set null` — so neither the TRUNCATE nor the `delete from auth.users` reaches it
`risk_rules`	6478	**NO**	No foreign keys at all
`risk_signals`	6498	**NO**	Only FK is to `risk_rules`, which is not truncated. `subject_id` is a plain `uuid` with no FK
`risk_reviews`	6539	**NO**	FKs to `profiles` and `risk_signals`, neither truncated

Note that `therapist_schedule_state` **is** in the list despite also being added later, so the list has been maintained for the roster work — the care-plan, evidence and risk tables were simply missed.

Why this is P1 rather than cosmetic. Three consequences, in increasing order of seriousness:

1. **The button lies.** "Deletes everything" is false for four tables.
2. **Orphaned evidence.** `communication_flags` rows survive with `author_id` and `patient_id` set to null by the cascade, so the flag remains but the people it names are gone. The table has a `BEFORE UPDATE OR DELETE` trigger that raises, so **there is no way to clear these rows short of adding the table to the TRUNCATE list** — a row delete is refused by design.
3. **The next test run is silently corrupted.** `risk_signals` carries a partial unique index giving at most one **open or reviewing** signal per `(rule, subject)`. A leftover open signal from a previous run holds that slot, so the same rule firing again on the same subject **writes nothing**. A tester executing `ADM-RISK-001` on a "clean" database sees an empty queue and reports a broken detector. This is exactly the leftover-state trap the plan warns about in §6.1 — except the plan tells the tester the reset protects them from it.

Fix. Add `communication_flags`, `risk_signals` and `risk_reviews` to the `TRUNCATE` list in a new `create or replace` of the function at the end of `schema.sql` (the file's own append-only convention). Decide `risk_rules` deliberately: it is configuration, so either truncate it and let the seed re-insert the eight rules, or reset it to defaults the way `site_settings` is — but do not leave it in its current state, where an edited threshold survives a wipe with nothing telling anyone.

Retest. `SETUP-RESET-001`, plus a new assertion: after a reset, `select count(*) from communication_flags` and `from risk_signals` both return `0`.

FIXED. `debug_reset_all_data` is redefined at the end of `schema.sql` (the file's append-only convention — later definitions win). `communication_flags`, `risk_signals` and `risk_reviews` are now named in the `TRUNCATE`, and `care_plans` / `care_plan_versions` / `contact_reveal_log` are named explicitly too rather

than relying on CASCADE, so a future foreign-key change cannot quietly take them back out of the reset. `risk_rules` is deliberately **not** truncated — it is configuration, like `site_settings` — and is instead reset to its seeded defaults, because an empty `risk_rules` would silently disable every detector rather than restoring it. `SETUP-RESET-001` now asserts both counts and the restored thresholds.

F-02 — `risk_reviews.reviewer_id` is `NOT NULL` with `ON DELETE SET NULL` · P2

Area. Plan §14.1, `ADM-RISK-002`. **Class.** VERIFIED-SOURCE.

Evidence. `schema.sql:6542`:

```
reviewer_id uuid not null references profiles(id) on delete set null,
```

These two clauses contradict each other. When the referenced profile is deleted, Postgres attempts to set `reviewer_id` to null, which the `NOT NULL` constraint refuses — so **the delete raises** rather than the reference being cleared. A scan of the whole schema found this is the **only** occurrence of the pattern; every other `on delete set null` column is nullable.

Reachability. Reviews are written by admins, and the reset preserves admins, so the standard reset path does not hit it. It becomes reachable when an admin account is deleted at all, and specifically when an admin who has reviewed a risk signal is demoted to another role and a reset then runs: the `delete from auth.users` cascades to `profiles`, the SET NULL fires, the constraint refuses, and **the entire reset transaction aborts** — leaving a half-tested database and an error a tester will not be able to interpret.

Fix. Pick the intent and state it. Either `reviewer_id` is nullable (a review outlives its reviewer, which matches how the other evidence tables treat authorship), or the reference is `ON DELETE RESTRICT` and a reviewer cannot be deleted while reviews exist. The current pair is neither, and produces a failure mode nobody chose.

FIXED. `alter table risk_reviews alter column reviewer_id drop not null`. A review outliving its reviewer is the intended reading — the same posture `communication_flags` already takes with `author_id`, whose reference is nullable for exactly this reason. The reviewer's name is resolved at render time, as it already is for a flag whose author has gone.

F-03 — `pain_assessments` is append-only by RLS alone, unlike its five siblings · P3

Area. Plan §12.4, `THR-HP-005`. **Class.** VERIFIED-SOURCE.

Evidence. `pain_assessments` is protected by `revoke update, delete on pain_assessments from authenticated` (`schema.sql:2212`). Five comparable tables use a **trigger** instead: `session_credit_ledger`, `care_plan_versions`, `communication_flags`, `contact_reveal_log`, `risk_reviews`.

Why the difference matters. The codebase already articulates the reason, in its own comment beside the ledger: *"The revoke covers a browser session; every route in this app writes with the service-role client, which bypasses RLS entirely. For a table whose whole value is that it cannot be rewritten, 'no route updates it' is not the same guarantee as 'an update raises'."* Pain Map data is clinical exam history whose value is precisely that a re-assessment is a new row, so a trend can be shown against the previous visit. It is protected one grade below the tables that share that property.

Severity is P3, not higher, because no route today updates it, and the gap is a consistency and defence-in-depth issue rather than a live hole.

Fix. Reuse the existing `communication_evidence_is_append_only()` trigger function — it already raises with the table's own name — and attach it to `pain_assessments`.

FIXED. `pain_assessments_no_change`, a `before update or delete` trigger using that same function. The table now carries the same guarantee as its five siblings.

F-04 — The plan overstates the admin scope guard (documentation defect) · P3

Area. Plan §4.4, §15.3, `ADM-SET-027`. **Class.** VERIFIED-SOURCE.

What the plan says. *"Every admin route guards with `requireAdminScope(section)`, not `getAdminUser()`."*

What is true. 92 of 95 admin routes call `requireAdminScope`. Three do not, and all three are deliberate and **stricter**:

Route	Guard	Why stricter
`admin/set-admin-scope`	`getAdminContext` + `scope !== "full"` → 403	A section check would let a `finance` admin widen its own access
`admin/debug-reset`	`getAdminContext` + `scope !== "full"` → 403	Gate 3 of the reset
`admin/create-account`	`getAdminContext` + full-only for minting an admin, full-or-operations otherwise	A scoped admin creating a full admin would hand itself the keys

Impact. No security impact — the exceptions are tighter than the rule. The defect is in the test plan: a tester reading §4.4 literally would file these three as violations. Correct the plan to state the rule as "every admin route guards on scope; three guard on `full` explicitly, because a section check would be too weak."

Fix applied in this session: the wording is corrected in the plan sources.

F-05 — The plan asserted a row cap on All Sessions that no longer exists (documentation defect) · P3 · Fixed

Area. Plan §14.2, `ADM-SESS-001`. **Class.** VERIFIED-SOURCE.

The plan asserted *"at most 200 rows are painted before a 'Show all' affordance appears"*. That was the **previous** design. The screen now uses the shared pager: `usePagedList(rows, { storageKey: "admin-sessions" })` with `DEFAULT_PAGE_SIZE = 10`. `AGENTS.md` records the change explicitly — *"Don't cap a list at an arbitrary number with a 'Show all' escape hatch: that was what All Sessions did, and 'Show all' then painted every row anyway."*

A tester would have reported a working screen as a defect. **Corrected in this session**; the test now asserts the pager, the per-browser page-size memory, and that exports and totals still run over the whole filtered set.

F-06 — The revenue split has no unit test · P1 (risk, not a defect)

Area. Plan §16. **Class.** EXECUTED (by absence) + VERIFIED-SOURCE.

`moneyByBucketFor` in `src/lib/adminMetrics.ts` is the only place the clinic's money is divided, and the plan's two identities live inside it. It feeds the Money summary strip, the tiles and the breakdown chart, so a defect there is wrong on three screens at once and wrong in the same direction, which makes it invisible to cross-checking between them.

There is no `adminMetrics.test.ts`. The nine test files cover the roster, home-visit pricing, the scanner, masking, settings, risk vocabulary, care plans and the consultation rule — every dependency-free module in `src/lib` **except** the one that computes the split. Its own comments record **four** distinct historical misstatements it has already been corrected for: counting uncompleted sessions in the therapist cut, dropping the travel fee into the clinic's share, subtracting refunds from a margin that had already had a cut taken, and collapsing "no hospital" with "hospital share unconfigured". Every one of those is exactly the kind of arithmetic a table-driven unit test pins down permanently.

This is not a defect — the current implementation reads correctly, and the identities hold by construction because the clinic share is computed as the difference rather than accumulated independently. It is the **largest untested risk surface in the application**, and it is the one the finance section of the test plan spends the most manual effort re-deriving by hand.

FIXED. `src/lib/adminMetrics.test.ts` — **24 tests**, structured around the four historical misstatements rather than around the function, so the point is that they stay fixed:

- both identities asserted, including on an empty range;
- a completed paid session earns a cut, a paid-but-undelivered one does not, and a cancelled-and-refunded one earns nothing while still counting in gross;
- a home visit's travel fee lands in the cut and never in gross, falls back to the online share when no home-visit rate is set, and treats a negative fee as zero rather than as a credit;
- the partner's commission is taken on net, so a refund reverses it, while the therapist's is untouched;
- "not referred" and "referred but unconfigured" stay apart — same money, opposite treatment;
- unpaid, slot-less and out-of-range sessions are ignored, and bucketing is by slot time rather than by when the money was taken;
- and the whole seven-row §16.1 reference dataset, so the figures a tester re-derives by hand on the Money screens are the figures pinned here.

The operational rates beside it (`computeNoShowRate`, `computeCancellationRate`, `computeRepeatBookingRate`) are covered too — including that they return **percentages, not fractions**, and `null` rather than zero when there is nothing to divide.

Original recommendation, for the record. Add `adminMetrics.test.ts` covering the §16.1 reference dataset: a completed paid session, a paid-but-not-completed one, a refunded one, a hospital-referred one, one with an unset therapist share, and a home visit with a travel fee — asserting both identities plus the excluded count. That dataset is already written and hand-computed in the plan; it converts directly into a test table. The manual finance tests then become a check on the *screens*, not on the arithmetic.

F-07 — `therapist_schedule_state` was read by the dashboard but never subscribed or published · P2 · Fixed

Area. Plan §14.0 (realtime), `ADM-TODAY-002`. **Class.** EXECUTED — this one was caught by the repository's own test suite during the live run, not by reading code.

What failed. `e2e/admin-multi-admin.spec.ts` H-006 asserts that every base table the admin dashboard queries appears in `ADMIN_REALTIME_TABLES`, and that everything in that list is added to the `supabase_realtime` publication. It failed with one name: `therapist_schedule_state`.

Why the lint check did not catch it. `scripts/check-realtime-coverage.mjs` and H-006 ask different questions. The script checks that every table the UI **subscribes to** is **published** — it passed, because this table was in neither list. H-006 checks that every table the dashboard **reads** is subscribed to, which is the stricter direction. Two checks, one gap between them.

Origin. The table arrived with the roster rebuild (`1aef991`), which added it, queried it from the dashboard at `page.tsx:327`, and updated the reset function's TRUNCATE list for it — but not the realtime arrays. It predates the changes in this branch.

Consequence, stated honestly. Benign but real. The dashboard reads `therapist_schedule_state` to hand the roster editor the version its next save must match. Without a subscription, one admin saving a roster leaves a second admin's open dashboard holding a stale version — the compare-and-swap catches it, so nothing is corrupted, but the second admin gets a 409 telling them to reload where a live refresh would simply have happened. It is a papercut, not a correctness hole; it is a finding because the codebase's own stated rule is that a new table goes into one of the two arrays **and** gets its `alter publication` line in the same change.

FIXED. Added to `ADMIN_REALTIME_TABLES` (the operational channel — a roster change is operational) and given the guarded `alter publication` block every other publication line in `schema.sql` uses. H-006 re-run: **passes**. `check:realtime` now reports 39 subscribed tables, all published.

F-08 — Eleven routes validated the request body before checking who was asking · P3 · Fixed

Area. Plan §18.2, `SEC-ROUTE-002`. **Class.** EXECUTED.

What was found. Calling 25 routes with no cookie and an empty body, 11 answered **400 with a field-validation message** rather than 401/403:

```
/api/appointments/cancel              400 {"error":"Missing appointmentId"}
/api/patient/condition-profile/submit  400 {"error":"Missing data"}
/api/razorpay/create-order            400 {"error":"Missing appointmentId"}
…and eight more
```

How serious, established rather than assumed. The obvious question is whether authentication was enforced *at all*, so the same 11 were re-called with well-formed bodies. Nine then returned 401/403. The two home-visit order routes validated more deeply first — `{"error":"A street address is required."}` — and only refused once a complete address was supplied. So **authentication was always enforced; nothing could be done anonymously.** No data leaked, no action was possible, and no response contained internals.

That makes this an **ordering** defect, not a hole — which is why it is P3 and not higher. It is still worth fixing on two grounds: an unauthenticated caller should not drive a route's parsing at all, and the other ninety-odd routes in this application already check auth first, so these eleven were the inconsistent ones.

FIXED. The `createClient` / `getUser` / `if (!user)` block was hoisted above body validation in all eleven, with a comment at each explaining why the order matters. `npm run verify` is green, and the five spec files covering those routes were re-run afterwards: **61/61 passed**, no regression. Re-running the probe: **11/11 refuse an anonymous caller**, and all 25 routes now refuse with nothing leaked.

---

5. What only a live run can establish

The following are **NOT-VERIFIABLE** here and carry real residual risk. They are the areas to run first when an environment exists.

Area	Why source inspection is insufficient
The whole payment funnel end to end	Signature verification, checkout rendering, callback timing and the webhook race are runtime behaviours. The mechanism is right; whether it fires correctly is untested
Concurrency (`PAY-CONC-001` , `THR-AVAIL-004` , `FIN-PAY-002`)	Row locks and CAS predicates are present in the SQL. Whether they actually serialise 12 concurrent reserves needs 12 concurrent reserves
Google Calendar / Meet	Neither the success path nor the capped-retry path can be exercised without credentials
Every screen rendering at all	A build proves it compiles, not that it paints. No page was loaded in this audit
Mobile at 390 × 844, keyboard nav, focus traps, reduced motion	Entirely runtime
Copy fidelity	The plan quotes route-handler strings verbatim; where a component re-words one before display, only a browser will show which the user sees
RLS behaving as written	Policies exist in the file. Whether the live project has them applied is a different question — and the schema-apply workflow is the only thing that closes it
The reset actually running	F-01 was found by reading the function. It should be confirmed by running it and counting rows

---

6. Verdict and recommended sequence

Pass, after remediation. The invariants the plan cares most about — the capture path, the ledger constraints, the append-only triggers, the split identities, the scope guards, the audit-log coverage — are all genuinely implemented where the plan says they are. **All six findings are fixed**, including the two documentation defects in the plan itself, and `npm run verify` is green on the result.

One thing still needs a person, not a commit. The schema changes here — the redefined reset function, the nullable `reviewer_id` , the new trigger, and the `auto_assign_therapist_enabled` column — reach a live database only when `supabase/schema.sql` is applied, either by hand with `node scripts/run-schema.mjs` or by the schema-apply workflow on a push to `main` . **Until that runs, the fixes exist in the file and not in the database**, which is the exact failure mode the schema conventions warn about: the app looks fixed in review and still behaves the old way in production. Apply it, then re-run `SETUP-RESET-001` and confirm the two counts come back zero.

Then run, in this order: the §16.3 payment-integrity sweep (highest value per hour, and entirely unverifiable statically), the §21 cross-role checks (they catch disagreements no single-role test can), then the §18 security sweep at the route level, then everything else in the plan's own recommended order.

---

7. Product review — flows worth changing

This section is written from a product rather than a QA seat. Each item names what the code does today, what it costs, and what changing it would buy. They are ordered by expected value, not by effort.

Seven of the nine shipped in the same pass as the findings. Each carries its outcome at the end. The two that did not are 7.6 and half of 7.7: both are owner decisions about *policy* rather than defects, and one of them (the ledger flip) is explicitly conditioned on evidence that does not exist yet.

#	Item	Outcome
7.1	Auto-assign a therapist at payment	**Shipped**, behind a switch, off for one release
7.2	Name who is waiting for a recommendation	**Shipped**
7.3	Payment reassurance on the first failure	**Shipped**
7.4	Webhook secret on System Health	**Shipped**
7.5	Sweeps only run when an admin looks	**Documented** — an ops change, not a code one
7.6	Patient approval queue	**Half shipped** — the screen now says what it decides; removing the gate is the owner's call
7.7	Two dark features	**Suggestions on by default. Ledger deliberately left off**
7.8	Smaller items	**Shipped** (page size, waiting-state date); therapist timezone still open

7.1 The funnel stalls on a manual admin step, and the data to remove it already exists · High value

Today. A patient picks a time, pays, and the appointment sits `requested` with `therapist_id = null` until an admin opens the dashboard and assigns someone. Only then is it `confirmed`, only then does a Meet link exist, and only then does the therapist see it. The roster — weekly schedule, exceptions, leave — is maintained in detail and deliberately does **not** feed the patient's picker.

What it costs. The gap between "patient has paid" and "patient has a confirmed session with a named clinician" is bounded by how quickly a human opens a browser. At a weekend or overnight booking that is hours. The patient's own screen says `Requested` with no therapist, which is the least reassuring possible state immediately after paying.

The change. Keep the picker unfiltered — that separation is deliberate and correct, and constraining what a patient may ask for is a different, worse product. But use the roster **server-side at payment confirmation**: if exactly one approved, active, non-on-leave therapist is free for that slot per the existing conflict check, assign them automatically and confirm. If zero or more than one, fall through to the queue exactly as today.

Why it is safe. Every component already exists — `therapistAvailability.ts`, `checkTherapistConflict.ts`, and the auto-assign path used by `bookPackageSession()`, which already assigns, confirms and mints a Meet link without an admin. This is wiring two existing mechanisms together, not new logic. The admin keeps full override.

Measure. Median minutes from `paid_at` to `status = 'confirmed'`, before and after.

SHIPPED. `src/lib/autoAssignTherapist.ts`, called from `/api/razorpay/verify` **and** `/api/razorpay/webhook` — both, so a patient who pays and closes the tab gets the same outcome as one who waits for the page, and neither path can develop its own idea of who is free. It reads the roster (weekly template, that date's exceptions, `on_leave`) plus the existing conflict check, and assigns only when **exactly one** eligible therapist is free, or when the patient's own `preferred_therapist_id` is among the free ones. Zero or two-or-more does nothing and the session waits in the queue exactly as before. It never throws. `decideAutoAssignment()` extracts that rule with the database taken out, so the judgement itself is unit-tested (10 tests) rather than only reachable through a payment. Gated by `auto_assign_therapist_enabled` — **off for its first release**, because it changes what happens to a booking after money has moved and the honest way to introduce that is with the previous behaviour one click away. `ADM-SET-021` covers all twelve cases.

7.2 Consultation → programme conversion depends on a therapist remembering · High value

Today. A programme can only be bought from a care plan, and a care plan can only be written from the session-note dialog of a **completed** session the therapist ran. The nudge is passive: a "Notes to write" figure on the therapist's Overview and a feed item. The detector that would measure the failure, `plan_conversion_low`, **ships disabled**.

What it costs. The entire revenue model above a single consultation runs through one optional action by a busy clinician at the end of a session. Nobody currently knows the conversion rate, because the rule that would compute it is off.

The change, in two steps and in this order. First **measure**: enable `plan_conversion_low` in a log-only posture for a month to establish the clinic's baseline — the reason it ships disabled is precisely that a threshold invented before a baseline fires on everyone or nobody, and that reasoning is right. Then, and only then, decide whether the prompt needs strengthening.

A cheap intervention that does not need the baseline: the session-note dialog already contains the recommendation panel. Make "no recommendation written" an explicit `needsYou` feed item with the patient's name and the date of the session, rather than an aggregate count. A named item is acted on; a number is not.

SHIPPED (the cheap half). The therapist's Overview feed now carries a named item per patient — *"QA Patient A is waiting to hear what next"* — pinned by `needsYou` and linking to that patient's chart. A patient with a live or already-purchased recommendation is excluded; a declined or withdrawn one is included, because that thread is open again. Capped at four, most recently seen first, beside the existing note nudge. `THR-CARE-006` covers it. **The measurement half is still the right first move** — `plan_conversion_low` remains disabled until the clinic has a baseline, and that reasoning has not changed.

7.3 The payment reassurance arrives two failures too late · Cheap, do it now

Today. After a failed or dismissed payment the wizard shows an error and re-labels the button **Pay ₹…**. **Now**. The reassuring message — *"Your booking is saved as pending — you can come back and pay any time from your dashboard"* — appears only after **three** failed attempts.

What it costs. A patient whose card fails once has no idea their booking survived. The most likely next action is to close the tab, and they have no reason to believe anything is waiting for them.

The change. Show the "your booking is saved" line on the **first** failure. Keep the escape-hatch link at three. One conditional; no new state.

SHIPPED. One conditional in `BookingWizard.tsx`: from the first failure the card reads *"Nothing was lost — your booking is saved and still held as unpaid. You can try again above, or pay later from your dashboard."* The amber escape hatch to the dashboard still waits until three, because that is the point at which retrying here has plainly stopped working. `PAT-PAY-003`.

7.4 Losing money silently when one environment variable is unset · Cheap, high consequence

Today. Without `RAZORPAY_WEBHOOK_SECRET`, `/api/razorpay/webhook` answers `503`. A patient who pays and closes the tab before the browser callback lands leaves a **paid Razorpay order against an unpaid booking**. Nothing anywhere reports this.

What it costs. Money collected with no session against it, discovered only when a patient complains or someone reconciles Razorpay by hand.

The change. Settings → System Health already exists and already surfaces sync failures and accounting disagreements. Add one row: webhook secret configured, yes or no, red when no. It is the only configuration whose absence silently loses money, and the screen that should say so is already built.

SHIPPED. A **Payment Confirmations** panel at the top of System Health. Configured, it states that a payment is confirmed by whichever arrives first, browser or webhook. Unconfigured, the whole panel turns red and says plainly that a patient who pays and closes the tab will leave a paid order against an unpaid booking with nothing flagging it, and names the variable to set. The presence of the secret is read server-side in the dashboard page and only the boolean crosses to the browser. `ADM-SET-030`.

7.5 Time-based work only happens when an admin is looking · Medium

Today. There is no cron, by design. The sweeps run at page render — and the audit confirms **the failed-Meet-sync retry and the risk detector run only on the admin dashboard render**. Package expiry additionally runs on the patient dashboard.

What it costs. On a quiet day where no admin opens the back office, no failed Meet link is retried and no risk signal is detected. The system's self-healing is coupled to somebody's browsing habits.

The change. Do not build a worker — the no-cron rule buys real simplicity and the sweeps are correctly bounded. Instead point a free uptime monitor at one authenticated-free endpoint every fifteen minutes. That is a configuration change with no code, and it converts "when an admin looks" into "every fifteen minutes" for the two sweeps that most need it.

NOT SHIPPED, deliberately — this one is an ops change, not a code change. Adding an endpoint that runs the sweeps would be a worker in all but name, and would undo the property that makes the no-cron design defensible: every sweep is bounded because it runs inside a render somebody is waiting for. The recommendation stands as written, for whoever configures the deployment.

7.6 Patient approval may be a queue that no longer earns its keep · Worth a decision

Today. Patients are gated on `approved`. But a genuine payment attempt auto-approves them. So the patient approval queue only ever contains people who registered **without** paying — and the plan's own §4.3 explains that this is deliberate, so that a failing card does not bounce someone to `/pending-approval`.

The question for the owner. What is the queue actually filtering? Everyone who pays is approved automatically; everyone who does not pay cannot book. The remaining population is people who registered to browse. If the answer is "nothing much", the queue is manual work with no decision attached to it, and it delays the one group who took a deliberate action but have not yet paid.

Two honest options. Keep it and state its purpose in one sentence on the screen so a new admin knows what they are deciding; or drop patient approval to "review only if flagged" and keep the human gate for therapists, where credential checking is a real decision.

HALF SHIPPED. The first option is done: Today → Approvals now states what it is deciding — *"Therapists here are waiting on a credentials check. Patients here registered without booking — a patient who starts a payment is approved automatically, so approving one from this list only affects what they can see, never whether they can pay."* **The second is not, and should not be done by an engineer.** Removing a gate on who can use a clinical product is a policy decision with a compliance dimension, and the sentence above is what makes it possible to take that decision with the facts in view.

7.7 Two features are built, dark, and rotting · Worth a decision

`therapist_suggestions_enabled` defaults **false**. `entitlement_ledger_authoritative` defaults **false**.

The first is the main re-booking mechanism — a therapist proposing the next session on a live programme — fully built, tested, and switched off. The second is a completed migration to an append-only credit ledger, with both systems written in parallel and a verification report on Settings → System Health.

Both defaults were right when written. Both now need a date rather than a default:

- **Suggestions:** launch it or delete it. A dark feature accumulates maintenance cost and drifts out of test coverage.
- **Ledger:** the parallel-write period is the risky state, not the destination — two sources of truth for session balances is precisely the condition the ledger was built to end. Once System Health has been clean over real traffic for an agreed window, flip it, then plan the removal of the counter writes. Carrying both indefinitely is the one outcome nobody chose.

7.8 Smaller items

Item	Today	Suggested
All Sessions page size	Defaults to 10 rows	**Shipped** at 25. The per-browser `storageKey` means anyone who already chose a size keeps it
A patient locked out of their own health profile	Read-only until a therapist writes the first record, with the CTA absent rather than disabled — correct, and deliberately quiet	**Shipped.** The waiting panel now names the session by date — *"Your session on 11/09/2026 is when this gets filled in"*, or after it *"Expected at your session on … If it still isn't here in a day or two, tell us and we'll chase it."* A patient with no session yet sees no date line rather than a placeholder
Therapist timezone	`profiles.timezone` is displayed on the roster and settable nowhere	**Still open** — it is a product decision (is this a one-timezone clinic?), not a defect, and is item 2 in the plan's own clarifications list. Either expose it on the therapist's profile or drop the column
`risk_rules` after a reset	Survives with edited thresholds (F-01)	Whatever is decided for F-01, make it deliberate — configuration that survives a wipe should do so for the same stated reason `site_settings` does

7.9 What is working well, and should not be traded away

A product review that only lists changes misrepresents the system. These are load-bearing decisions that are correct and should survive future pressure to "simplify":

- **One capture path.** `record_payment_capture` being a single idempotent database function is why duplicate webhooks, gateway retries, a webhook racing a callback and a double-clicked Pay button are

all safe without any of them knowing about each other. Do not add a second fulfilment path; extend its `purpose` check.

- **A therapist picks a package, never a price.** "The therapist set their own price" is not a policy someone enforces — it is a thing the schema cannot express. That is a much stronger guarantee than a rule, and it costs nothing.
- **Evidence tables append-only by trigger, not RLS.** Every route writes with the service role, which bypasses RLS entirely; a record the evidenced party could edit is not evidence. The distinction is subtle and correct.
- **A flag is never an accusation.** The risk queue deliberately carries no action buttons, and stores the row ids behind a finding rather than a score. That is what makes running heuristics over clinical data defensible at all.
- **The roster does not filter the patient's picker.** It reads like an omission and is a decision. Anyone proposing to "fix" it is proposing a product change with a deploy-sized blast radius.